GUARDIAN EYE Multi-Modal Real-Time Monitoring System

Full System Technical Documentation

---

1. System Overview

Guardian Eye is a sophisticated edge-computing platform designed to intercept emergency events in real-time through three primary modes: **Vision (Fall Detection and Gestures)**, **Acoustic (Voice Keywords)**, and **Hardware (GSM Failback)**.

The python-based hardware engine processes all ML logic locally on the edge (e.g., a Raspberry Pi) and streams real-time state changes up to a centralized Firebase Firestore cloud. This cloud data is instantly syndicated down to a Progressive Web Application (PWA) built on React/Vite, triggering critical push-alerts and modern 3D UI overlays on a remote Guardian's dashboard.

---

2. Essential System Architecture & Files

The project is strictly separated into two architectural pillars:

- **Python Edge Engine**: The local sensor fusion software running continuously on the hardware device via Python multiprocessing.
- **Web Dashboard**: An isolated `website/` React application configured for mobile installation and real-time visualization.

Core Backend Files:

- `main.py` - The Process Orchestrator and main initialization sequence.
- `fall.py` / `fall_process.py` - YOLO-based Vision Analysis Engine.
- `voice_process.py` - Acoustic Background Intelligence.
- `decision_engine.py` - State-machine integrating both video and audio.
- `gsm.py` - Hardware SIM card integration for offline SMS.

- `service_account.json` - Firebase Admin SDK authentication keys.
- `device_config.json` - Local (git-ignored) configuration containing the device's mapped Firebase `user_id` and optional Gemini API key.

---

3. Deep Dive into Edge Processing Components

`main.py` (The Orchestrator)

`main.py` is the execution root of Guardian Eye. Because performing computer vision and hot-word voice detection simultaneously on a single CPU thread would cause severe video lag, `main.py` utilizes Python's `multiprocessing` library. 1. It creates two separate physical CPU processes: one for the `fall_process` and one for the `voice_process`. 2. It generates shared memory event queues (`event_queue` and `voice_queue`). 3. It continually reads incoming data from these overlapping queues and executes the `EmergencyDecisionEngine` loop.

`fall_process.py` & `fall.py` (Vision Engine)

This is a highly optimized camera ingestion pipeline. It grabs frames from `cv2.VideoCapture(0)` or a native Raspberry Pi `picamera2/gstreamer` pipeline. Each fractional split-second frame is routed directly into `fall.py`, which instantiates the `SimpleHighAccuracyFallDetector`.

`gesture_process.py` (Gesture Component)

Gesture logic utilizes the same exact mathematical keypoint array extracted during the Fall routine. By analyzing the proximity of `Wrist` keypoints `(9, 10)` relative to `Shoulder` keypoints `(5, 6)`, the engine mathematically dictates if the user's arms are "Crossed", "Raised", or held over their "Chest".

`voice_process.py` (Acoustic Engine)

Running cleanly in the background, this module leverages Python `speech_recognition` and either raw `pocketsphinx`, `vosk`, or `whisper` logic (depending on the Pi's configuration) to actively listen to the microphone. When decibel thresholds are crossed, it converts audio buffers into raw strings and compares them against emergency keyword lexicons (like "Help", "Fire", "Pain").

`decision_engine.py` (Sensor Fusion)

The `EmergencyDecisionEngine` receives concurrent data streams. It establishes logic rules using internal timers. For example:

- If Vision detects `FALL` for over 5 seconds -> TRIGGER ALERT
- If Vision detects `GESTURE(Hand over Chest)` AND Audio detects `Help` -> TRIGGER CRITICAL ALERT
- If Audio detects `Fire` 3 times in 10 seconds -> TRIGGER ALERT

`gsm.py` (Offline Failover)

Uses direct AT/Serial commands (`/dev/serial0` at `9600 baud`) to communicate directly with hardware SIM800L modules. If the local device loses WiFi routing to Firebase, `gsm.py` instantly dispatches an SMS message to the phone number configured in `GUARDIAN_EMERGENCY_PHONE`.

---

4. Machine Learning Models Utilized

A. YOLOv8 Pose Estimation (`yolov8n-pose.pt`)

This model is the absolute foundational core of the Guardian Eye logic. It does not just draw boxes; it natively maps **17 distinctly identifiable biological keypoints** (eyes, ears, shoulders, hips, knees, ankles) on the human body.

- **Fall Detection Logic**: By comparing the coordinates of Keypoints 5 & 6 (Shoulders) versus Keypoints 11 & 12 (Hips), the angle of the human torso is mathematically calculated. If the angle approaches 90° (horizontal) and the bounding box Aspect Ratio (width > height) inverts, a fall is suspected.

B. YOLOv8 Standard Object Detection (`yolov8n.pt`)

- **New False-Positive Suppression Feature**: To prevent Guardian Eye from alarming when someone simply decides to lie down on their bed, a second Object Detection model is conditionally executed. When a fall is highly suspected, this secondary model rapidly checks the frame for COCO IDs 56, 57, and 59 (Chairs, Couches, Beds). If the horizontal human's center X/Y coordinates are overlapping inside the bounding box of a Bed, the alarm is completely canceled, and the condition is classified as "Resting."

---

5. End-To-End Synchronization (Edge to Cloud)

When `decision_engine.py` conclusively confirms an alert timeline, it immediately triggers the `push_emergency_to_firebase` protocol. 1. The script accesses `service_account.json` to gain secure, direct backend access to Google Cloud servers. 2. It looks at `device_config.json` to know specifically which `user_id` it belongs to. 3. It performs an asynchronous write targeting: `Firestore DB > users/{user_id}/emergencies/{auto_id}`. 4. It attaches a rigorous JSON dictionary containing the severity level, reason (e.g., "Fall & Keyword Detected"), Unix Timestamp, and sets `status = "active"`.

---

6. Guardian Eye Web Dashboard (React/Vite Infrastructure)

The `website/` folder is a distinctly modern, Progressive Web Application (PWA).

Modern Aesthetics & 3D Integration

The front-end extensively uses `TailwindCSS` for responsive layouts and glassmorphism. It implements advanced geometric 3D models using `@react-three/fiber` and `@react-three/drei` dynamically intersecting with `framer-motion` enter animations on the `Landing.jsx` introduction sequence.

Real-Time Firebase Fetching

`Dashboard.jsx` does NOT use simple HTTP GET REST requests. Because emergency data must be instantaneous, it utilizes a Firebase **`onSnapshot` WebSocket Listener**. 1. An open socket is maintained to `users/{user_id}/emergencies`; ordered by the newest timestamp. 2. The exact millisecond the Python script pushes the alert to Firebase, the WebSocket receives the packet delta. 3. If the packet's status is `"active"`, `Dashboard.jsx` instantly transitions the UI out of its "System Secure" green orb into a massive pulsing red `Action Overlay` preventing the user from navigating away until acknowledged.

Acknowledging Emergenices

When the user physically clicks the "Acknowledge & Dismiss" button on the UI, the React framework pushes an `updateDoc` command back to Firebase, flipping the document's record to `status: "resolved"`. This satisfies the cloud state, which instantly clears the red action overlay.

Progressive Web App (PWA)

Constructed using `vite-plugin-pwa`, the React layout compiles a `manifest.json` and local service workers. Users loading the website on a Chrome or Safari mobile browser can natively click "Add to Home Screen," which installs the Dashboard as a fully functional, standalone application complete with its custom icon.